Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

INF205
Ressurseffektiv programmering

3 Datastrukturar

3.1 Grafdatastrukturar
3.2 Kode-data-samutvikling
3.3 Listedatastrukturar
3.4 Grafalgoritmar
3.5 Romlege domene

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

Underveisevaluering

INF205

4. mars 2026

Langsiktig vidareutvikling

Emnerevisjonen må drøftast innan 11. mars 2026.

Anbefalingane dykkar kan bidra til dette. De kan fokussere på det som står i emnebeskrivinga, men òg gjerne tenkje vidare. **Gjerne tenk veldig fritt!**

Biblioteket For studenter Norwegian English

Norges miljø- og biovitenskapelige universitet

INF205

Ressurseffektiv programmering

Studiepoeng:
5

Ansvarlig fakultet:
Fakultet for realfag og teknologi

Emneansvarlig:
Martin Thomas Horsch

Campus / nettbasert:
Undervises campus Ås

Undervisningens språk:
Norsk

Frekvens:
Årlig (vårsemester, første halvdel)

Forventet arbeidsmengde:
125t = 24t forelesninger + 12t datalab + 89t selvstendig arbeid

Undervisnings- og vurderingsperiode:
Emnet blir tilbudt i vårparallellen. Emnet har undervisning/vurdering heilt gjennom den første halvdelen av vårparallellen.

Langsiktig vidareutvikling

Emnerevisjonen må drøftast innan 11. mars 2026.

Anbefalingane dykkar kan bidra til dette. De kan fokussere på det som står i emnebeskrivinga, men òg gjerne tenkje vidare. Gjerne tenk veldig fritt!

5 minutt idéutvikling

egian English

Norges miljø- og biovitenskapelige universitet

INF205

Ressurseffektiv programmering

Studiepoeng:
5

Ansvarlig fakultet:
Fakultet for realfag og teknologi

Emneansvarlig:
Martin Thomas Horsch

Campus / nettbasert:
Undervises campus Ås

Undervisningens språk:
Norsk

Frekvens:
Årlig (vårsemester, første halvdel)

Forventet arbeidsmengde:
125t = 24t forelesninger + 12t datalab + 89t selvstendig arbeid

Undervisnings- og vurderingsperiode:
Emnet blir tilbudt i vårparallellen. Emnet har undervisning/vurdering heilt gjennom den første halvdelen av vårparallellen.

Langsiktig vidareutvikling

Emnerevisjonen må drøftast innan 11. mars 2026.

Anbefalingane dykkar kan bidra til dette. De kan fokussere på det som står i emnebeskrivinga, men òg gjerne tenkje vidare. Gjerne tenk veldig fritt!

5 minutt idéutvikling

10 minutt i grupper: Formuler eitt tiltak

Norges miljø- og biovitenskapelige universitet

INF205

Ressurseffektiv programmering

Studiepoeng:
5

Ansvarlig fakultet:
Fakultet for realfag og teknologi

Emneansvarlig:
Martin Thomas Horsch

Campus / nettbasert:
Undervises campus Ås

Undervisningens språk:
Norsk

Frekvens:
Årlig (vårsemester, første halvdel)

Forventet arbeidsmengde:
125t = 24t forelesninger + 12t datalab + 89t selvstendig arbeid

Undervisnings- og vurderingsperiode:
Emnet blir tilbudt i vårparallellen. Emnet har undervisning/vurdering heilt gjennom den første halvdelen av vårparallellen.

Kva kan vi realisere dette semesteret?

Dei frie tankane kan me ikkje nødvendigvis realisere i det løpande kurset.

No skal det tenkje meir konkret om kva som er umiddelbart mogleg.

Materiale:

- Eksempel & andre delte filer
- Glossardokument: Nøkkelomgrep og føreslegne definisjonar

Pensumlitteratur:

- B. Stroustrup, *A Tour of C++*, 3. utgåve, Pearson Education (ISBN 978-0-13-68
- B. Stroustrup, H. Sutter (eds.), C++ Core Guidelines, **2015-2025**.

Struktur:

1. Moderne C++
   - Slides: Fyrste samling (4.2.) [video], språket C++, peikarar (5.2.) [... gs C++ (12.2.)
   - Problemliste 1, problemliste A
   - Litteratur:
     - C. E. Leiserson, N. C. Thompson, J. S. Emer, B. C. Kuszmaul, B. W. La ... r Moore's law?", *Science* **368**(6495): eaam9744, doi:10.1126/science.aam9744
2. Minnehandtering
   - Slides: Stakk og haug (13.2.), vanlege minnehandteringsfeil (19.2
   - Problemliste 2, problemliste B, problemliste C
3. Datastrukturar
   - Slides: Grafdatastrukturar (27.2.), samutvikling av kode og data (4.3.)
   - Underveisevaluering (open til 5.3.)
   - Litteratur:
     - M. Poveda Villalón, S. Chávez Feria, S. Carullo Pérez, R. García Castro, "Towards a UML-based notation for OWL ontologies", in *Proceedings of VOILA23*: 2, CEUR-WS, **2023**
     - B. Stroustrup, "Concept-based generic programming in C++", arXiv:2510.08969 [cs.PL], doi:10.48550/arXiv.2510.08969, **2025**
4. Parallelprogrammering

Diskuter i gruppene (10 minutt):

I kurset er det to veker att (veke 11 & 12), er det konkrete anbefalingar til den delen?

Kva med videomaterialet? Er dette det riktige når det gjeld innhaldet? Når det gjeld formatet? Har de konkrete forslag?

Nettskjema for underveisevaluering

https://nettskjema.no/a/inf205-2026v

Anbefaling: Lagre det de skriv som frie tekstbidrag, slik at de kan gjenbruke det under emneevalueringa.

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

3 Datastrukturar

3.1 Grafdatastrukturar

3.2 Kode-data-samutvikling

INF205

4. mars 2026

Ordliste

Data ingest	Datainntak
Entity-relationship diagram (ER diagram)	Entitet-relasjonsdiagram (ER-diagram)
Individual (synonymous with «object»)	Individ (synonymt med «objekt»)
Relationship	Forhold
Relationship type	Forholdstype

Designing classes: Entity-relationship diagrams

particular: **object** entity individual

universal: **class** entity type concept

(sometimes: attribute)
property

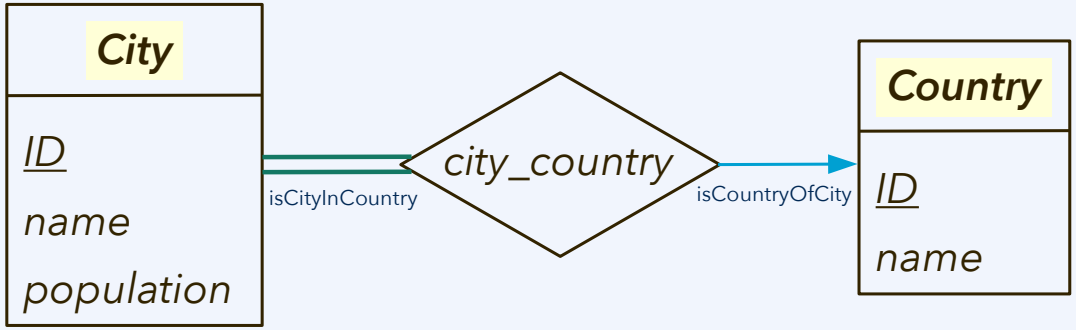
(sometimes: attribute type)
attribute
(in OWL: DatatypeProperty)

	City
long, unique	*ID*
std::string	*name*
int	*population*

	Country
int, unique	*ID*
std::string	*name*

Designing classes: Entity-relationship diagrams

particular:	**object** (entity, individual)	**relationship**	**property** (sometimes: attribute)
universal:	**class** (entity type, concept)	**relation** (relationship type) (in OWL: ObjectProperty)	**attribute** (sometimes: attribute type) (in OWL: DatatypeProperty)

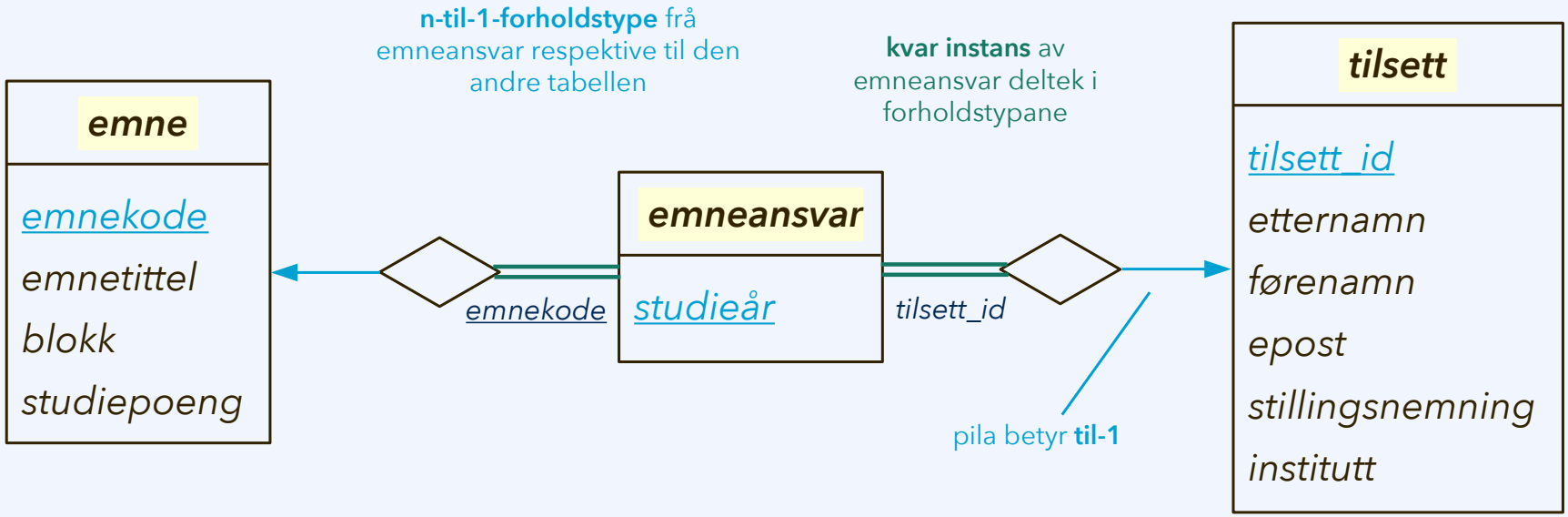


"every City is in such a relationship"

"it is an *N*-to-1 relation from Cities to Countries"

Konseptuelt skjema: Entitet-relasjonsdiagram

ER-diagram kan bli brukt til å spesifisere både eit databaseskjema og klassestrukturen til eit objektorientert program.



Obs: Ofte blir diamantane i ER-diagram kalla for «relasjonar». Men det er forvirrande, sidan det allereie er i bruk som alternativ teknisk term for «tabell». Vi skal referere til diamantane som **forholdstypar**.

Entity-relationship (E-R) diagrams

particular: object (entity, individual) relationship property (sometimes: attribute)

ID	name	population
wd:Q172	Toronto	2731571
wd:Q1490	Tokyo	14047594
wd:Q585	Oslo	693494
wd:Q1761	Dublin	553165
wd:Q1781	Budapest	1723836
wd:Q2807	Madrid	3305408
wd:Q60	New York City	8804190
wd:Q240	Brussels-Capital Region	1218255
wd:Q1842	Luxembourg	128512
wd:Q1757	Helsinki	643272
wd:Q1754	Stockholm	978770
wd:Q1748	Copenhagen	644431
wd:Q270	Warsaw	1790658

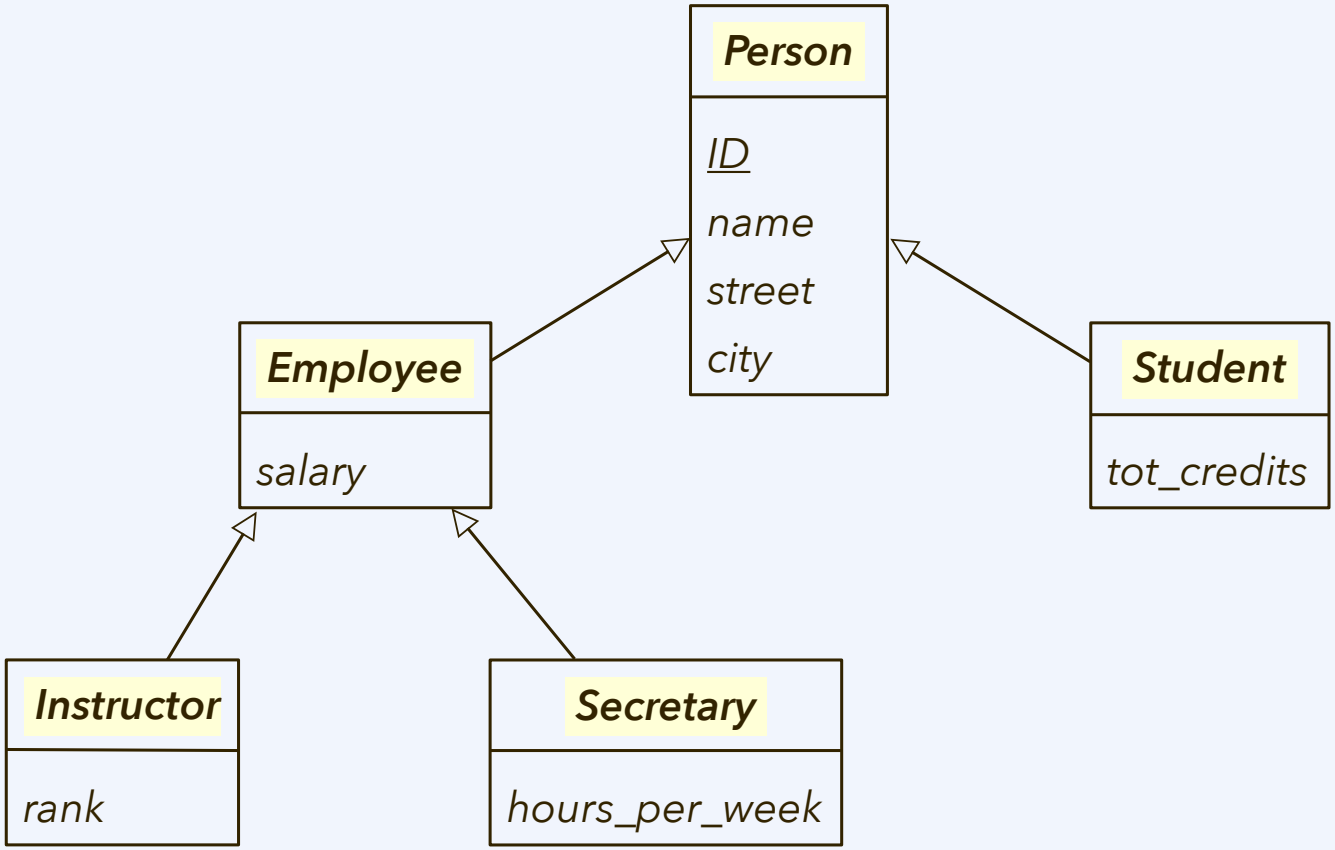
City entity set

ID	name
wd:Q16	Canada
wd:Q17	Japan
wd:Q20	Norway
wd:Q27	Republic of Ireland
wd:Q28	Hungary
wd:Q29	Spain
wd:Q30	United States of America
wd:Q31	Belgium
wd:Q32	Luxembourg
wd:Q33	Finland
wd:Q34	Sweden
wd:Q35	Denmark
wd:Q36	Poland

Country entity set

Taksonomi (klassehierarki)

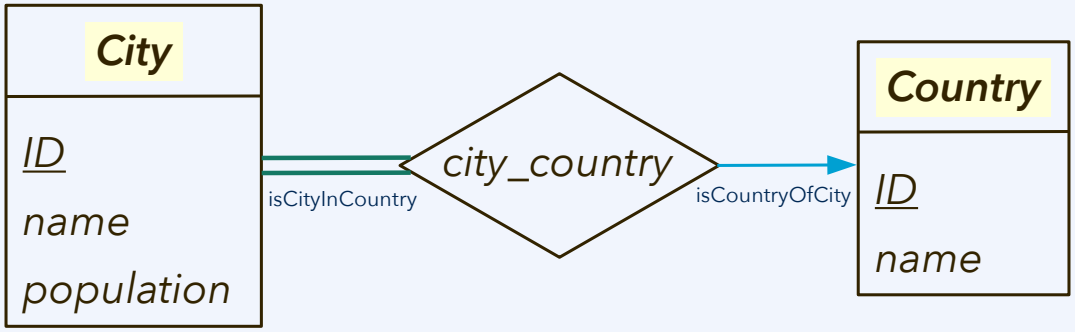
Eksempel frå Silberschatz *et al.*[1] (Fig. 6.18):



[1]A. Silberschatz, H. F. Korth, S. Sudarshan, *Database System Concepts*, 7th int. stud. edn., McGraw-Hill, **2019**.

Designing classes: Entity-relationship diagrams

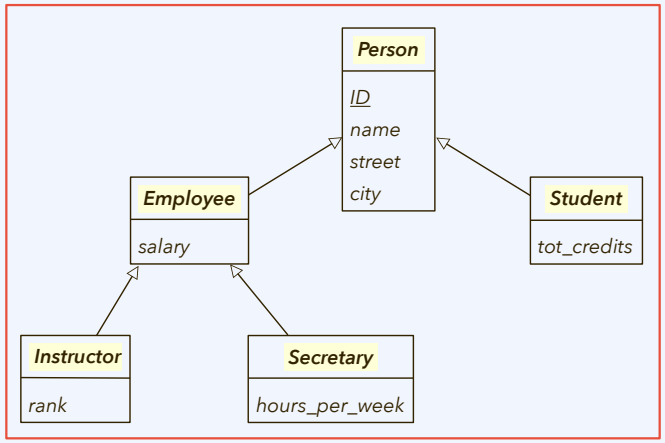
particular:	**object** (entity, individual)	**relationship**	**property** (sometimes: attribute)
universal:	**class** (entity type, concept)	**relation** (relationship type) (in OWL: ObjectProperty)	**attribute** (sometimes: attribute type) (in OWL: DatatypeProperty)



"every City is in such a relationship"

"it is an *N*-to-1 relation from Cities to Countries"

This was also an entity-relationship diagram:



More on entity-relationship diagrams:

- Silberschatz *et al.*, *Database System Concepts*, Chapter 6
- https://en.wikipedia.org/wiki/Entity-relationship_model

Implement relations using non-owning pointers

By storing a pointer to object B as a property of object A, we can encode the relationship between A and B, so that methods from A can access B.

This can go both ways, if needed. Then B also has a pointer to A as a property:

```
class City
{
 public:
   City(string in_name, int in_population, Country* in_country);
   …

 private:
   long ID;
   string name;
   long population;

   Country* country;
};
```

```
class Country
{
 public:
   Country(string in_name);

   void add_city(City* c);
   ...

 private:
   int ID;
   string name;
   vector<City*> cities;
};
```

Example file: city-country.zip

Implement relations using non-owning pointers

By storing a pointer to object B as a property of object A, we can encode the relationship between A and B, so that methods from A can access B.

This can go both ways, if needed. Then B also has a pointer to A as a property:

```
class City
{
 public:
   City(string in_nam
   …

 private:
   long ID;
   string name;
   long population;

   Country* country;
};
```

Den løysinga bruker ikkje-eigande peikarar (non-owning pointers), men det *kan* òg vera aktuelt med eigarskap.

Kva skjer no når levetida til eit Country-objekt går ut?

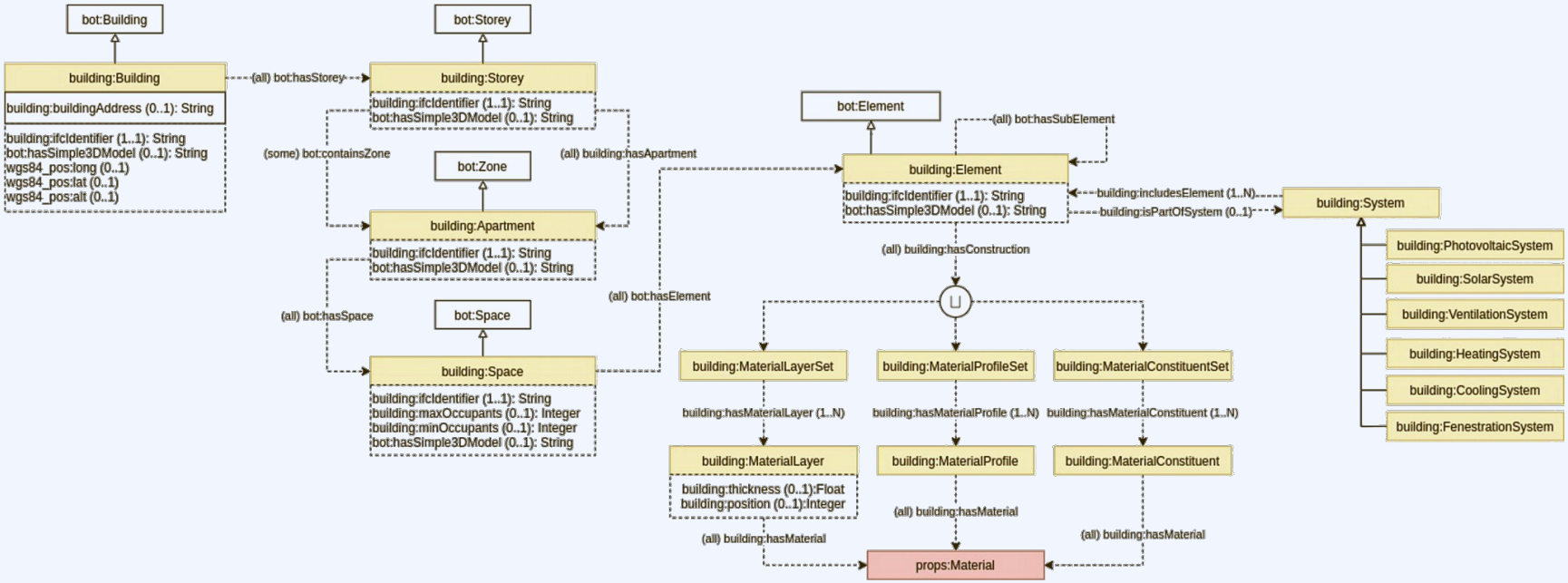
```
class Country
{
 public:
   Country(string in_name);

   void add_city(City* c);
   …

 private:
   int ID;
   string name;
   vector<City*> cities;
};
```

Example file: **city-country.zip**

E-R diagrams on draw.io and Chowlk[1, 2]

The draw.io tool can be used for E-R diagrams using a variety of conventions.

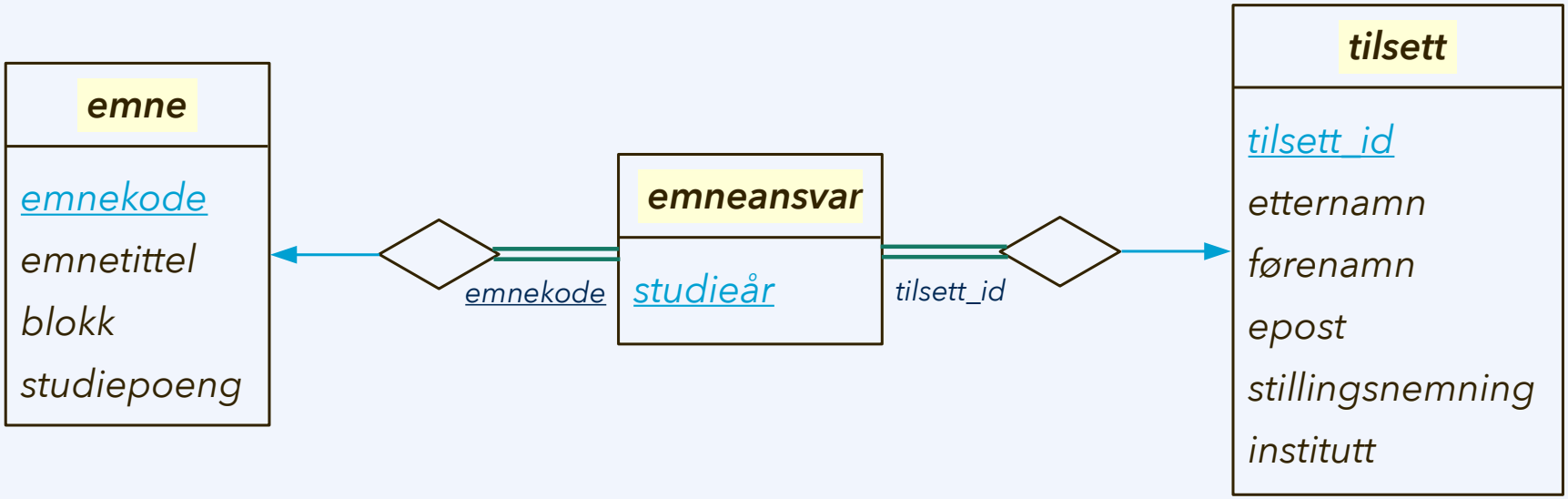
With Chowlk by Poveda Villalón *et al.*,[1, 2] these can be converted to ontologies.



[1]M. Poveda Villalón *et al.*, in *Proc. VOILA23*, *CEUR Works. Proc.* **3508**: 2 (link to paper), **2023**.

[2]Chowlk template: https://chowlk.linkeddata.es/static/resources/chowlk-library-complete.xml
Lightweight version: https://chowlk.linkeddata.es/static/resources/chowlk-library-lightweight.xml

Difference between E-R use in databases and OOP



In a *database schema*, objects are identified through *primary keys* (IDs), and their relationship to others are give by *foreign keys*, *i.e.*, other objects' IDs. For example, above, an "emneansvar" entry contains a "tilsett_id" number.

In OOP, we *can* do this the same way, with a registry of objects, *e.g.* using a std::map. But normally, that would be poor style. The standard way of doing this is: Holding a pointer or object reference to the object as a property.

JSON in digital infrastructures

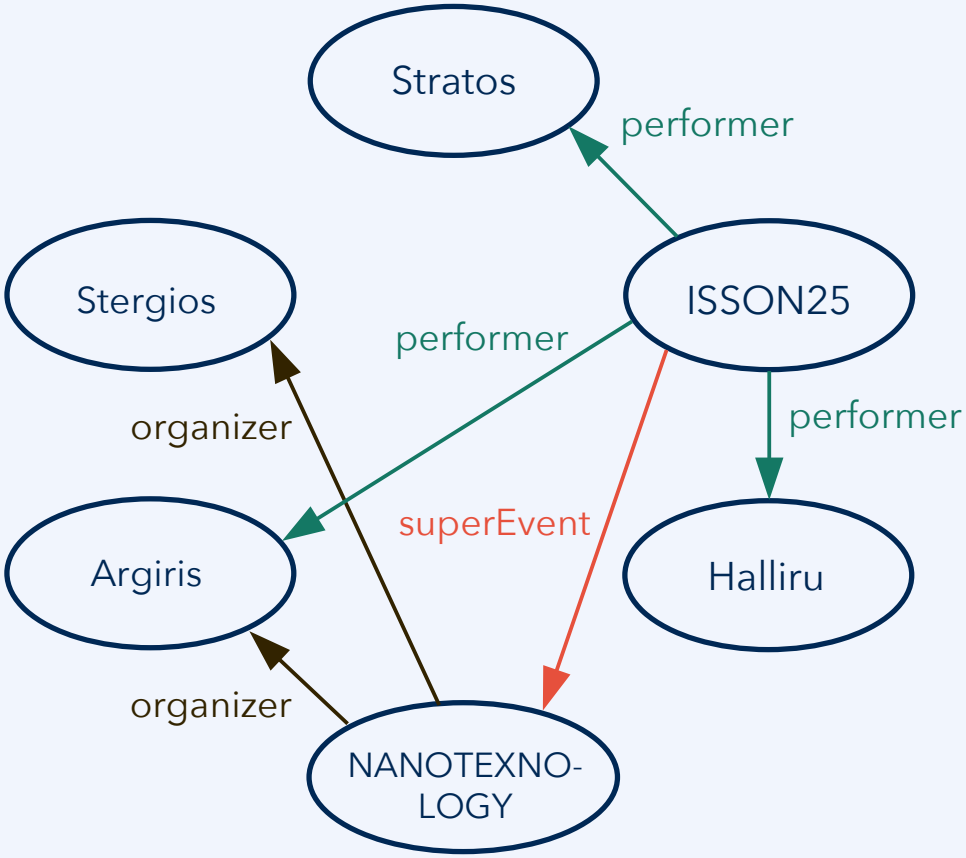
JSON is often used for data ingest & extraction into/from DBs via RESTful APIs:

- JSON is a hierarchical format, in which one element can *contain* other elements; in this sense it is equivalent to XML, but with less overhead.
- JSON can be used as a type in *relational databases* including MySQL,[1] *i.e.*, JSON formatted data can be ingested without transformation.
- It is also used in *non-relational DBs*; *e.g.*, MongoDB is based on JSON.[2]
- The hierarchical structure – *transitivity* of the containment relation – limits the way in which objects can be connected to each other (*trees* only).
- Data in a JSON file are self-contained, there is no standard way to include *external resources*; except via "JSON linked data" (JSON-LD).

[1] https://dev.mysql.com/doc/refman/8.0/en/json.html

[2] https://docs.mongodb.com/guides/server/introduction/

Utveksling av objektdata i JSON-format



hierarkisk strukturert (tre)

ikkje kopla til semantisk vev

```
{
  "label": "ISSON25",
  "performer": [
    {
      "label": "Halliru"
    }, {
      "label": "Stratos"
    }, …
  ],
  "superEvent": {
    "label": "NANOTEXNOLOGY",
    "organizer": [
      {
        "identifier": "Argiris"
      }, {
        "identifier": "Stratos"
      }
    ]
  }
}
```

Eksempelfil: **konferanse.json**

JSON for lenkja data: JSON-LD

JSON: JavaScript-objektnotasjon

JSON-LD: Gyldig JSON med ID-ar på semantisk vev, og lenkjer mellom dei

```
{
    "@context": {
        "example-abox": "https://www.nanotexnology.com/index.php/isson/data#",
        "schema": "https://schema.org/"
    },
    "@id": "example-abox:ISSON25",
    "@type": "schema:EducationEvent",
    "schema:performer": [
        {
            "@id": "example-abox:Halliru_Ibrahim",
            "@type": "schema:Person"
        }, {
            "@id": "example-abox:Argiris_Laskarakis",
            "@type": "schema:Person"
        }, …
    ],
    "schema:superEvent": {
        "@id": "example-abox:NANOTEXNOLOGY_2025",
        "@type": "schema:BusinessEvent",
        "schema:organizer": [
            {
                "@id": "example-abox:Argiris_Laskarakis"
            }, {
                "@id": "example-abox:Stergios_Logothetidis",
                "@type": "schema:Person"
            }
        ]
    }
}
```

IRI-ar som tillèt grafstruktur

Instansiering: IRI-ar for klasser (typar)

Eksempelfil: **konferanse-lenkja-data.json**

JSON-LD: Verktøy og bruk til annotering

JSON: JavaScript-objektnotasjon
JSON-LD: Gyldig JSON med ID-ar på semantisk vev, og lenkjer mellom dei

JSON-LD Playground (https://json-ld.org/playground/)
Google's Schema Markup Validator (https://validator.schema.org/)
Google's Rich Results Test (https://search.google.com/test/rich-results)

Kode

```
1  {
2     "@context": "https://schema.org/",
3     "@id": "https://home.bawue.de/~horsch/tea
   /dat121/",
4     "@type": "EducationEvent",
5     "name": ["Introduction to data science", "Datavitskap
   innføringsemne", "Datavitenskap innføringsemne"],
6     "alternateName": "DAT121",
7     "startDate": "2023-08-14",
8     "url": "https://home.bawue.de/~horsch/teaching
   /dat121/",
9     "location": {
10       "@id": "https://ror.org/04a1mvv97",
11       "@type": "CollegeOrUniversity",
12       "name": ["Norwegian University of Life Sciences",
   "Noregs miljø- og biovitskaplege universitet", "Norges
   miljø- og biovitenskapelige universitet"],
13       "alternateName": "NMBU",
14       "location": {
15          "@id": "http://www.wikidata.org/entity/Q54062",
16          "@type": "City",
17          "name": "Ås",
18          "alternateName": "Aas"
19       },
20       "url": "https://www.nmbu.no/",
21       "sameAs": [
22          "http://www.wikidata.org/entity/Q1725075",
23          "https://isni.org/isni/000000040607975X"
24       ]
```

Testresultater > Aktiviteter DEL

Manglende felt «eventAttendanceMode» (valgfritt)	
Manglende felt «performer» (valgfritt)	
Manglende felt «endDate» (valgfritt)	
Manglende felt «description» (valgfritt)	
id	https://home.bawue.de/~horsch/teaching/dat121/
type	EducationEvent
name	Introduction to data science
name	Datavitskap innføringsemne
name	Datavitenskap innføringsemne
alternateName	DAT121
startDate	2023-08-14
url	https://home.bawue.de/~horsch/teaching/dat121/
location	
Manglende felt «address» (valgfritt)	
id	https://ror.org/04a1mvv97

CRC cards: Class – responsibilities – collaborations

In structured or procedural programming, it is comparably easy to analyse a code even on a line-by-line basis and make statements about the execution state. These can take the form of **invariants** (conditions that are always met).

Classes in object oriented programming have a much more flexible control flow, as their methods can be called at any time and in any order. It can help to make the invariants explicit during design, *e.g.*, using **CRC cards** as a tool.

class

Country	
<u>ID</u> *name* *population* *register a city* *deallocate the country*	***City***

responsibilities collaborations

Country
Population of the country is the sum of cities' populations. (Requires update when changing city population.) *Each city needs a country. (If deallocated, move cities to "null country".)*

(backside can be used for invariants)

Generic programming[1, 2]

Extensive reliance on templates is also called **generic programming** (GP), which can be seen as its own programming paradigm, building on object-oriented programming but going beyond it; "by implementing programs generically, a single implementation can be used for many different types".[1]

We have seen: C++ supports such design by (1) inheritance and (2) templates.

From C++20 onward, **concepts** are introduced as GP language constructs. They describe requirements for a type (*e.g.*, it must provide an operator such as "<<", a particular method, or we must be able to add it to an integer, …).

```
// old style: does not make clear what
// we expect from the class SeqnT
template<typename SeqnT, …>
  void test_sequence(SeqnT* sqn, …)
{ … }
```

```
// new style, where we would define Sequence
// as a concept (and not as an abstract class)
template<Sequence SeqnT, …>
  void test_sequence(SeqnT* sqn, …)
{ … }
```

[1] L. Escot, J. Cockx, Proc. *ACM Prog. Lang.* **6**: 625–649, doi:10.1145/3547644, **2022**.
[2] B. Stroustrup, «Concept-based generic programming in C++», arXiv:2510.08969 [cs.PL], **2025**.

Modern C++ constructs for ownership

By including <**memory**> from the C++ standard library, encapsulated pointer templates (**smart pointers**) can be used that can support secure manual memory management:

```
std::unique_ptr<T> p = new T();
```

Now the unique_ptr p has taken ownership of the new T object.
When p is deallocated, its destructor ensures that the T object is deallocated.

Unique-pointer objects *cannot be copied*, they can only be *moved*!

A smart pointer that can be copied, so that multiple smart pointers can point to the same object and "own it together," is of type **std::shared_ptr<T>**. That object is deallocated once all shared pointers to it have been deallocated.

Core Guidelines I.11 recommends against ever passing ownership through "raw" pointers or references. It advises to rely on smart pointers instead.

Glossar

Entitet-relasjonsdiagram

Forhold

Forholdstype

Glossar

Entitet-relasjonsdiagram

Definisjon: «modell for å beskrive datastrukturer i databaser som danner en logisk beskrivelse av databasens innhold og oppbygning» (termportalen.no)

Forhold

Definisjon: elementært fakt om konkrete objekt

Eit forhold instansierer ein forholdstype.

Forholdstype

Definisjon: tilknyting av klasser som beskriv korleis individ som tilhøyrer dei respektive klassene, kan forhalde seg til einannan; tilsvarar som regel eit verb i menneskeleg språk

T.d. kan "er forfattar av" ta subjekt som tilhører klassen "person" og objekt som instansierer "publikasjon". I så fall er "person" domenet og "publikasjon" kodomenet til forholdstypen "er forfattar av".

Noregs miljø- og biovitskaplege universitet

Institutt for datavitskap

Digitalisering på Ås

INF205
Ressurseffektiv programmering

3 Datastrukturar

3.1 Grafdatastrukturar
3.2 Kode-data-samutvikling
3.3 Listedatastrukturar
3.4 Grafalgoritmar
3.5 Romlege domene